

Backend Documentation

Syllabus Chatbot - FastAPI + LangChain Agent Architecture

Stack: Python | FastAPI | LangChain | MongoDB Atlas | Mistral + Gemini | GitHub MCP

1. Overview

The backend is an AI chatbot service for an ECE department. It answers questions about the college syllabus (from stored course PDFs), calculates CGPA, performs live web searches, and operates the user's GitHub account (repos, issues, pull requests, files).

How it works in one sentence

A question arrives at one REST endpoint, a two-layer router decides what kind of question it is, binds only the relevant tools to an LLM, and an agent loop executes the LLM's chosen tools until it can stream a final answer back.

Key components

- FastAPI server (uvicorn) - HTTP layer, CORS, health check
- Hybrid router - regex first (free/fast), Gemini Flash LLM only when ambiguous
- Agent loop - up to 4 rounds of LLM tool-calling per question
- Local tools - CGPA math, syllabus RAG search, Tavily web search
- GitHub MCP client - 32 remote tools over one authenticated connection
- MongoDB - stores embedded syllabus chunks (vector search) and chat history

2. Folder Structure

```
backend_langchain/
|-- server.py           Entry point: uvicorn on 127.0.0.1:8000 (auto-reload)
|-- .env               Secrets: API keys, GitHub PAT, MongoDB URL
|-- requirements.txt    Dependencies
`-- app/
    |-- main.py         FastAPI app: startup sequence, CORS, /health
    |-- core/config.py   Reads .env ONCE into global `settings` object
    |-- routes/llm.py    POST /chatbot - the ONLY chat endpoint
    |-- auth/
    |   |-- clerk.py     Verifies Clerk JWT tokens (login security)
    |   |-- dependencies.py get_current_user - blocks unauthenticated calls
    |-- ai/
    |   |-- router.py     Decides WHAT the question is about (intent)
    |   |-- chat_service.py The brain: runs the whole agent pipeline
    |-- tools/
    |   |-- __init__.py   Tool registry: merges local + GitHub tools
    |   |-- cgpa_tool.py  calculate_cgpa (local math)
    |   |-- rag_tool.py   syllabus_rag_search (searches course PDFs)
    |   |-- tavily_tool.py web_search_tool (live web search)
    |-- services/mcp_client.py GitHub MCP connection (32 tools)
    |-- rag/             One-time setup: PDFs -> chunks -> embeddings -> Mongo
    `-- db/mongodb.py    Async MongoDB connection (motor)
```

Supporting folders: venv/ (Python environment), outputs/ (generated files served statically), uploads/, plus log files server.log and server.err.log.

3. Startup Workflow

Triggered by uvicorn loading `app.main:app`. The lifespan context manager in `main.py` runs the sequence below before the server accepts requests.

Step	Component	What happens
1	<code>server.py</code>	uvicorn starts <code>app.main:app</code> on port 8000 with <code>reload=True</code>
2	<code>main.py</code> lifespan	<code>connect_to_mongo()</code> opens the async Motor client
3	<code>mcp_client.connect()</code>	Connects to <code>https://api.githubcopilot.com/mcp/</code> using <code>GITHUB_API_KEY</code> as Bearer token. Retries 3 times with exponential backoff (2s, 4s, 8s)
4	MCP handshake	Fetches all tool schemas from GitHub, keeps the 32 essential ones (<code>search_repositories</code> , <code>get_file_contents</code> , <code>issues</code> , <code>PRs</code> , <code>commits</code> ...)
5	<code>invalidate_tool_cache()</code>	Clears cached tool registries so they rebuild including MCP tools
6	Ready	GET <code>/health</code> reports <code>{mcp_connected, mcp_tools_count}</code>

Failure handling

- If all 3 startup attempts fail, the server still boots - GitHub tools stay empty.
- `ensure_connected()` performs a lazy reconnect on the next GitHub query (throttled to once per 30s after failures; successes reset the throttle).
- A lock serializes concurrent reconnect attempts so parallel chats cannot stampede the endpoint.

4. Request Workflow (POST `/chatbot`)

Example query used throughout this section: 'give readme file of Instagram-clone project in my github'.

Step	File	What happens
1. Auth	<code>routes/llm.py</code> + <code>auth/</code>	Frontend POSTs JSON <code>{user_prompt, ...}</code> with a Clerk JWT. Depends(<code>get_current_user</code>) verifies the token against Clerk's JWKS URL before anything runs.
2. Setup	<code>chat_service.py</code>	ChatService created with prompt + conversation history. Initializes three LLMs: Mistral Small (primary), gemini-flash-latest (fallback), Gemini Flash temperature=0 (router).
3. Route L1	<code>router.py</code>	Regex patterns score every intent (greeting, cgpa, syllabus, web_search, github). Score ≥ 2 routes directly; score 1 or 0 goes to Layer 2. Example matches 'github' 3x -> high confidence github.
4. Route L2	<code>router.py</code>	Only for ambiguous queries: Gemini Flash re-classifies using the last 6 history messages and picks intent + tool names as JSON.
5. Tool binding	<code>chat_service.py</code>	Intent selects a tool subset. GitHub intent binds ONLY the 32 MCP tools (local tools excluded to save tokens). <code>bind_tools(..., tool_choice='auto')</code> .
6. Guard	<code>chat_service.py</code>	GitHub intent but zero MCP tools? Tries <code>mcp_client.ensure_connected()</code> (lazy reconnect) before giving up with a clear error message.

Step	File	What happens
7. Agent loop	chat_service.py	Up to MAX_TOOL_ROUNDS=4: invoke LLM -> if tool_calls present, execute them concurrently (asyncio.gather), append ToolMessages, loop. Otherwise treat response as final answer.
8. Streaming	routes/llm.py	Final answer streams token-by-token to the frontend as a StreamingResponse.

Worked example - tool call chain

```

Round 1: get_me()
        -> username: priyadharshiniannadurai413-dev
Round 2: search_repositories(query='user:<username> instagram clone in:name')
        -> repo found: Instagram-clone
Round 3: get_file_contents(owner=..., repo='Instagram-clone', path='README.md')
        -> README content returned
Final:   LLM formats and streams the README to the user

```

5. Intent Routing Details

Layer 1 regex triggers (router.py)

Intent	Priority	Example keywords matched
greeting	2	hi, hello, thanks, who are you (whole-message match only)
cgpa	5	calculate, cgpa, gpa, sgpa, grade, semester gpa
syllabus	4	syllabus, subject, unit, credits, semester, lab, cbc, 22XXnnn codes
web_search	3	latest, placement, news, gate, salary, company, 2024-2026
github	10	github, repo(sitory), issue, PR, commit, branch, merge, fork, clone, push, pull, code, readme, license, gist, milestone, starred

Ties are broken by priority - github wins because its keywords often collide with other intents (e.g. 'code', 'branch' can appear in academic questions).

Confidence rules

- score >= 2: high confidence - route immediately, no extra LLM cost.
- score == 1: low confidence - Gemini Flash router refines using conversation history.
- score == 0: ambiguous - LLM router decides between syllabus/web_search/etc.
- Greeting short-circuits instantly with a canned reply - zero LLM calls.

6. Tools Reference

Local tools (tools/ folder)

Tool	Purpose	Bound by intents
calculate_cgpa	CGPA/GPA arithmetic from semester GPAs	cgpa
syllabus_rag_search	Vector search over embedded course PDFs in MongoDB	syllabus, ambiguous
web_search_tool	Tavily API live web search	web_search, ambiguous

GitHub MCP tools (32, grouped)

Group	Tools
Repos (7)	search_repositories, create_repository, fork_repository, get_file_contents, create_or_update_file, delete_file, push_files
Branches (2)	list_branches, create_branch
Commits (3)	list_commits, get_commit, search_commits
Issues (8)	list_issues, issue_read, issue_write, add_issue_comment, search_issues, list_issue_fields, list_issue_types, sub_issue_write
Pull Requests (10)	list_pull_requests, pull_request_read, create_pull_request, update_pull_request, update_pull_request_branch, merge_pull_request, pull_request_review_write, add_reply_to_pull_request_comment, request_copilot_review, search_pull_requests
Search + User (2)	search_code, get_me

tools/__init__.py merges local + MCP tools into cached registries (get_all_tools, get_tool_registry, get_mcp_tool_registry). Any MCP change calls invalidate_tool_cache() so registries rebuild on next access.

7. Natural-Language GitHub Resolution

Users rarely paste repo links - they say 'readme of my Instagram-clone project'. The system prompt (_SYSTEM_PROMPT in chat_service.py) teaches the LLM a mandatory workflow so it never guesses owner/repo names:

```
User mentions repo WITHOUT owner/link?
1. get_me() -> learn authenticated username (owner)
2. search_repositories(keywords) -> fuzzy-find candidate repos
3a. exactly ONE match -> use full_name for the real request
3b. MULTIPLE matches -> list them, ask user to choose
3c. NO match -> tell user repo not found

Full URL given (github.com/owner/repo)?
-> extract owner+repo directly, skip steps 1-2

NEVER invent owner or repo names.
```

Router support: the github regex also fires on readme/license/gist/milestone/starred, so questions like 'show the README of my portfolio' route correctly even without the word 'github'.

8. Recent Fixes & Reliability Notes

Fix	Problem before	Solution
MCP retry + lazy reconnect	One failed connection at startup killed GitHub tools until manual restart	connect() retries 3x with exponential backoff; ensure_connected() lazily reconnects on demand, throttled after failures, concurrency-safe via asyncio.Lock
Repo resolution workflow	Questions without explicit repo links failed or hallucinated owners	System-prompt workflow: get_me -> search_repositories -> act (section 7)
Fallback model retired	gemini-1.5-flash returned 404 (model retired), so Mistral rate-limits produced 'technical difficulties' with no safety net	Switched fallback to gemini-flash-latest - a version-less alias that always resolves to Google's current flash model
Router keyword gaps	Queries like 'show README of X' missed the github intent	Added readme, license, gist, milestone, starred to the Layer-1 regex and LLM-router rules

Operational notes

- Rate limits: Mistral free tier throttles bursts; the Gemini fallback absorbs overflow automatically.
- Token economy: GitHub intent binds only MCP tools (~12k tokens instead of ~28k); essential-tools filter drops non-core schemas.
- Security: rotate the GitHub PAT if it was ever exposed; .env is loaded once at import time - restart after changing it.
- Health probe: GET /health returns mcp_connected and mcp_tools_count for quick diagnostics.